

10. IMAGE COMPRESSION

AIM:

To perform image compression using blockwise DCT by thresholding coefficients and reconstructing the image with inverse DCT. To evaluate compression efficiency by calculating Compression Ratio and Redundancy.

INTRODUCTION:

Image compression is both an art and a science that focuses on reducing the amount of data required to represent an image without significantly compromising its quality. It is one of the most impactful and widely used technologies in the field of digital image processing. Every day, billions of images and videos are compressed and decompressed seamlessly—whether in digital cameras, on the web, or while streaming high-quality media. The techniques used make this process practically invisible to end users, while saving enormous amounts of storage and transmission bandwidth.

the compression ratio is:

$$C = \frac{b}{b'}$$

and the redundancy is:

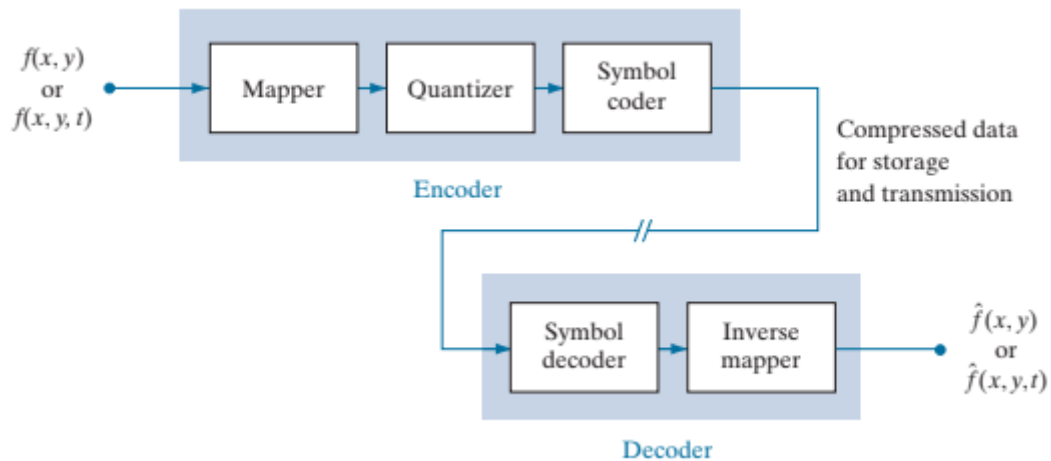
$$R = 1 - \frac{1}{C}$$

Coding Redundancy Occurs when fixed-length binary codes are used to represent intensity values without considering their probabilities. Since some intensities occur more frequently than others, assigning shorter codewords to common values (as in Huffman coding) reduces redundancy compared to natural binary coding.

Spatial and Temporal Redundancy Neighboring pixels in an image often have similar intensities, meaning their values are correlated. Similarly, in video sequences, adjacent frames are temporally correlated. Storing these values independently leads to unnecessary repetition. Techniques like run-length coding, predictive coding, and transforms (such as DCT) exploit this redundancy.

Irrelevant Information Images often contain details that are imperceptible or unimportant to the human visual system. By removing or approximating such details, storage can be reduced without a noticeable loss of quality. Quantization in transform coding is an example of exploiting this type of redundancy.

A general image compression system has two parts: an encoder and a decoder. The encoder removes redundancies in three steps:



The decoder performs the reverse operations—symbol decoding and inverse mapping—to reconstruct the image. If quantization is used, the reconstruction may not perfectly match the original, leading to lossy compression. If quantization is omitted, the system achieves lossless compression, where the output matches the original image exactly.

CODE: COMPRESSION OF IMAGE (MATLAB)

```
clc;
clear;
close all;

I = imread('cameraman.tif');
[r, c] = size(I);
out = zeros(r, c);

for i = 1:8:r
    for j = 1:8:c
        blk = double(I(i:i+8-1, j:j+8-1));
        dct = dct2(blk);
        dct(abs(dct) <= 40) = 0;
        rec = idct2(dct);
        out(i:i+8-1, j:j+8-1) = rec;
    end
end
out = uint8(out);
subplot(1,2,1);

imshow(I);
title('Original Image');

subplot(1,2,2);
imshow(out);
title('Reconstructed ');
```

OUTPUT:



Figure: (a) Original grayscale image (b) Reconstructed image after blockwise DCT compression with thresholding in matlab

CODE: COMPRESSION OF IMAGE (PYTHON)

```
import cv2

import numpy as np

from scipy.fftpack import dct, idct

import matplotlib.pyplot as plt

u=cv2.imread(r"C:\Users\DELL\Pictures\IPfolder\IPLAB_2025\DIP3E_Original_I
mages_CH02\Fig0222(a)(face).tif", cv2.IMREAD_GRAYSCALE)

r0, c0 = u.shape

out = np.zeros_like(u, dtype=float)

# Process 8x8 blocks
for i in range(0, r0, 8):
    for j in range(0, c0, 8):
        blk = u[i:i+8, j:j+8].astype(float)

        # 2D DCT (apply along rows and cols)
        temp = dct(dct(blk.T, norm='ortho').T, norm='ortho')

        # Thresholding (set small coefficients to zero)
        temp[np.abs(temp) <= 40] = 0
```

```

# 2D inverse DCT

g = idct(idct(temp.T, norm='ortho').T, norm='ortho')

# Put block back into output

out[i:i+8, j:j+8] = g

# Clip and convert to uint8
out = np.clip(out, 0, 255).astype(np.uint8)

plt.subplot(1, 2, 1)
plt.imshow(u, cmap='gray')
plt.title("Original")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(out, cmap='gray')
plt.title("Reconstructed")
plt.axis("off")

plt.show()

```

OUTPUT:



Figure: (a) Original grayscale image (b) Reconstructed image after blockwise DCT compression with thresholding in matlab

CODE: TO COMPUTE COMPRESSION RATIO AND REDUNDANCY

```
import cv2
import numpy as np
from scipy.fft import dct, idct
import matplotlib.pyplot as plt

# Read grayscale image
u=cv2.imread(r"C:\Users\DELL\Pictures\IPfolder\IPLAB_2025\DIP3E_Original_Images_CH02\Fig0222(a)(face).tif", cv2.IMREAD_GRAYSCALE)
r0, c0 = u.shape
out = np.zeros_like(u, dtype=float)

B = 8      # Block size
T = 40     # Threshold value
# Counters for compression ratio
total_coeffs = 0
nonzero_coeffs = 0

# Process 8x8 blocks
for i in range(0, r0, B):
    for j in range(0, c0, B):
        blk = u[i:i+B, j:j+B].astype(float)
        # Apply 2D DCT
        dct_block = dct(dct(blk.T, norm='ortho').T, norm='ortho')
        # Count coefficients before thresholding
        total_coeffs += dct_block.size
        nonzero_coeffs += np.count_nonzero(np.abs(dct_block) > T)

# Compute Compression Ratio and Redundancy
CR = total_coeffs / nonzero_coeffs
R = 1 - (1 / CR)

print(f"Compression Ratio (CR): {CR:.2f}")
```

```
print(f"Redundancy (R): {R:.2f}")
```

OUTPUT:

```
Compression Ratio (CR): 29.91  
Redundancy (R): 0.97
```

INFERENCE:

The blockwise DCT compression effectively reduces the image data by setting small coefficients to zero, while preserving the visual quality of the reconstructed image. The experiment shows that most image information is concentrated in a few significant DCT coefficients, allowing high compression without noticeable loss.

CONCLUSION:

Blockwise DCT with thresholding is an efficient method for lossy image compression. It reduces storage requirements and transmission bandwidth while maintaining acceptable image quality. The approach highlights the importance of selecting an appropriate threshold to balance compression and fidelity. DCT-based compression is a practical and widely used technique in digital image processing for efficient image representation.